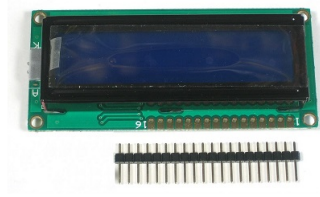
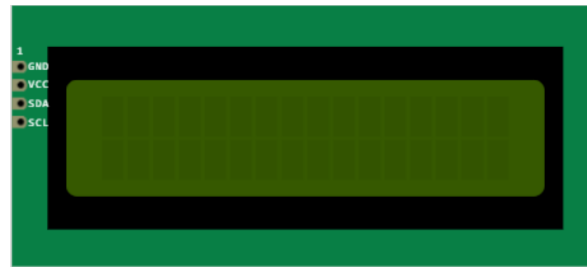


LCD 2X16

Real

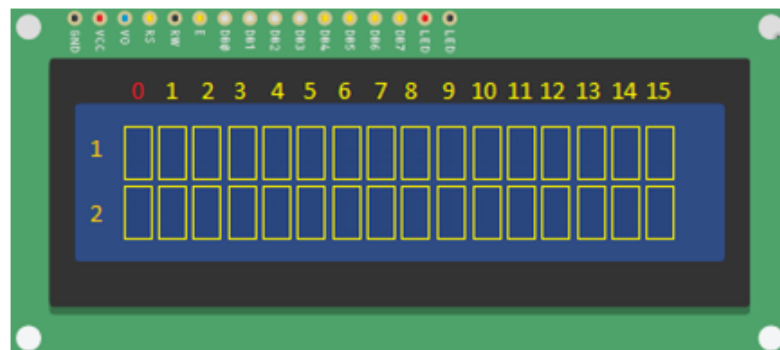


Simulador



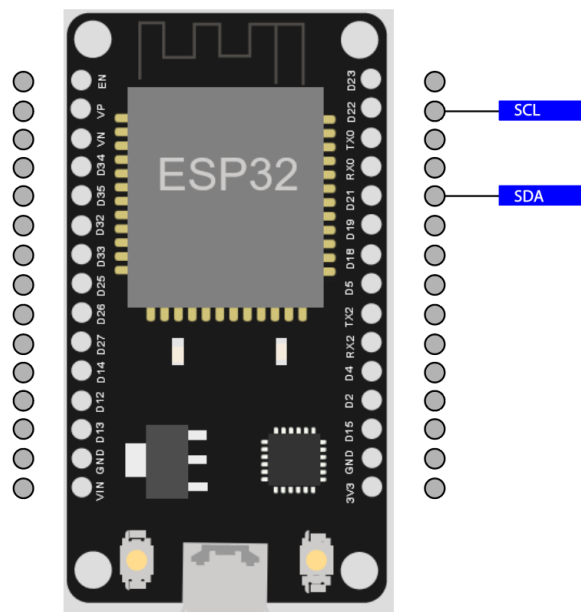
Representación de filas y columnas

Real



Pines del ESP32

ESP32 – DevKit V1 - 30 PINES - SIMULADOR - PINES I2C



LIBRERIAS EXTERNAS

PASO 1 - OBTENEMOS LA LIBRERÍA EXTERNA Y LA CARGAMOS EN WOKWI

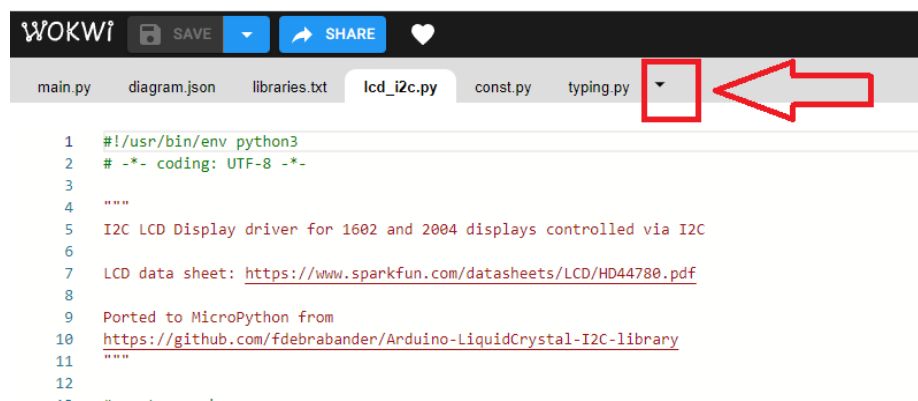
Wokwi tiene la posibilidad de cargar librerías externas, para realizar esta acción agregamos una solapa nueva en la cual pegamos el o los codigos que nos provee la librería.

En este caso estamos cargando tres archivos de la librería:

<https://github.com/brainelectronics/micropython-i2c-lcd/tree/main>

- **lcd_i2c.py**
- **const.py**
- **typing.py**

Pestaña desde la que agregamos nuevos archivos



PASO 2 - Revisamos como se están importando los módulos

En el caso de la importación de los módulo en ocasiones debemos revisar como se están invocando, por ejemplo para que esta librería funcione se debe modificar la forma en la cual **const.py** y **typing.py** son importadas desde **lcd_i2c.py** de esto:

```
# custom packages
from . import const as Const

# typing not natively supported on MicroPython
from .typing import List, Optional, Tuple, Union
```

A esto:

```
# custom packages
import const as Const

# typing not natively supported on MicroPython
from typing import List, Optional, Tuple, Union
```

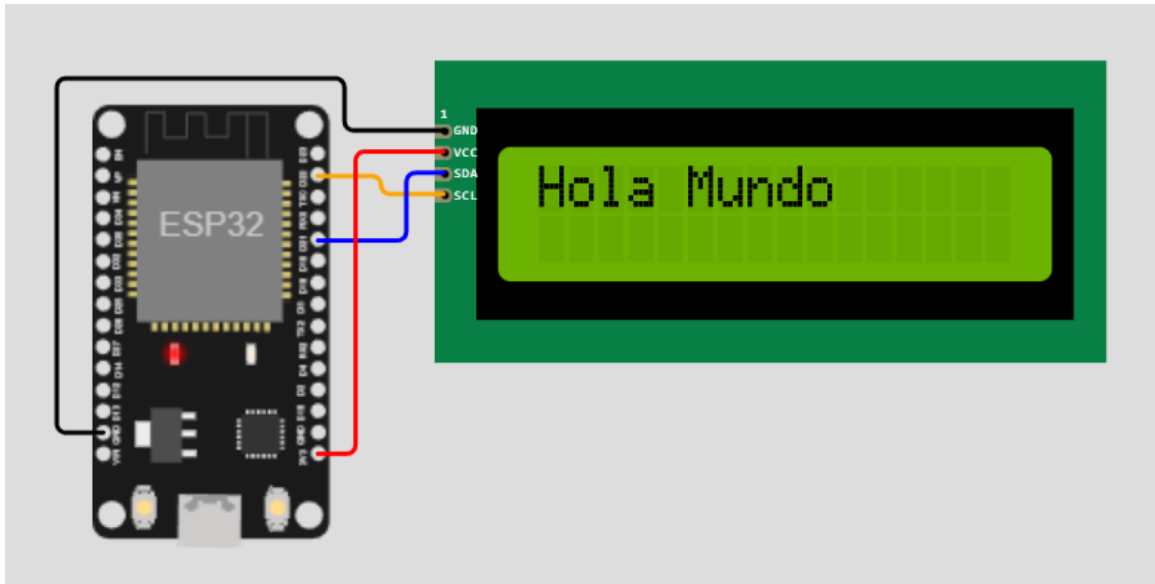
Ejemplo en simulador

PASO 3 - Primer ejemplo - Hola Mundo!

</pre>

MONTAJE

Pestaña desde la que agregamos nuevos archivos



PASO 3.1 - Importamos Librerías

```
from machine import Pin
from machine import I2C
from lcd_i2c import LCD
```

PASO 3.2 - Indicamos salidas y filas y columnas del lcd

```
sda=Pin(21, Pin.OUT)
scl=Pin(22, Pin.OUT)

NUM_ROWS = 2
NUM_COLS = 16
```

PASO 3.3 - Obtenemos la dirección del LCD en decimal, hexadecimal y binario

```
i2c=I2C(0, sda=sda, scl=scl, freq=400000)

decimal_i2c=i2c.scan()
hexadecimal_i2c=hex(decimal_i2c[0])

def decimal_a_binario(decimal):
    binario=0
    i=0
    while(decimal>0):
        digito=decimal%2
        decimal=int(decimal//2)
```

```

        binario=binario+digito*(10**i)
        i=i+1
    return binario

binario_i2c=decimal_a_binario(decimal_i2c[0])
print("Dirección del lcd en decimal, hexadecimal y binario")
print("Decimal: ", decimal_i2c, "Hexadecimal: ", hexadecimal_i2c, "Binario: ", binario_i2c, end="\n\n")

```

PASO 3.4 - Configuramos el LCD

```

lcd = LCD(addr=int(hexadecimal_i2c), cols=NUM_COLS, rows=NUM_ROWS, i2c=i2c)

lcd.begin()
lcd.print("Hola Mundo")

```

Código completo

```

from machine import Pin
from machine import I2C
from lcd_i2c import LCD

sda=Pin(21, Pin.OUT)
scl=Pin(22, Pin.OUT)

NUM_ROWS = 2
NUM_COLS = 16

##### Obtenemos la dirección del LCD

i2c=I2C(0, sda=sda, scl=scl, freq=400000)

decimal_i2c=i2c.scan()
hexadecimal_i2c=hex(decimal_i2c[0])

def decimal_a_binario(decimal):
    binario=0
    i=0
    while(decimal>0):
        digito=decimal%2
        decimal=int(decimal//2)
        binario=binario+digito*(10**i)
        i=i+1
    return binario

binario_i2c=decimal_a_binario(decimal_i2c[0])
print("Dirección del lcd en decimal, hexadecimal y binario")
print("Decimal: ", decimal_i2c, "Hexadecimal: ", hexadecimal_i2c, "Binario: ", binario_i2c, end="\n\n")

```

CONFIGURAMOS I2C

```
lcd = LCD(addr=int(hexadecimal_i2c), cols=NUM_COLS, rows=NUM_ROWS, i2c=i2c)

lcd.begin()
lcd.print("Hola Mundo")
```

Generación de caracteres especiales

Nosotros podemos generar un carácter de nuestro agrado e incorporarlo al display, tengamos en cuenta que no pueden superar los 5 u 8 caracteres especiales en un mismo momento en el LCD dependiendo del LCD. Notar como el valor de 1 representa cuadro lleno y el cero cuadro vacío.

Caracter		Valor Binario
	4 3 2 1 0	76543210
0		0b00001100
1		0b00010010
2		0b00010010
3		0b00001100
4		0b00000000
5		0b00000000
6		0b00000000
7		0b00000000
8		0b00000000

Implementación en wokwi

```
lcd.create_char(
  location=0,
  charmap=[0x00, 0x00, 0x11, 0x04, 0x04, 0x11, 0x0E, 0x00]
```

```
# this is the binary matrix, feel it, see it
# 00000
# 00000
# 10001
# 00100
# 00100
# 10001
# 01110
# 00000
```

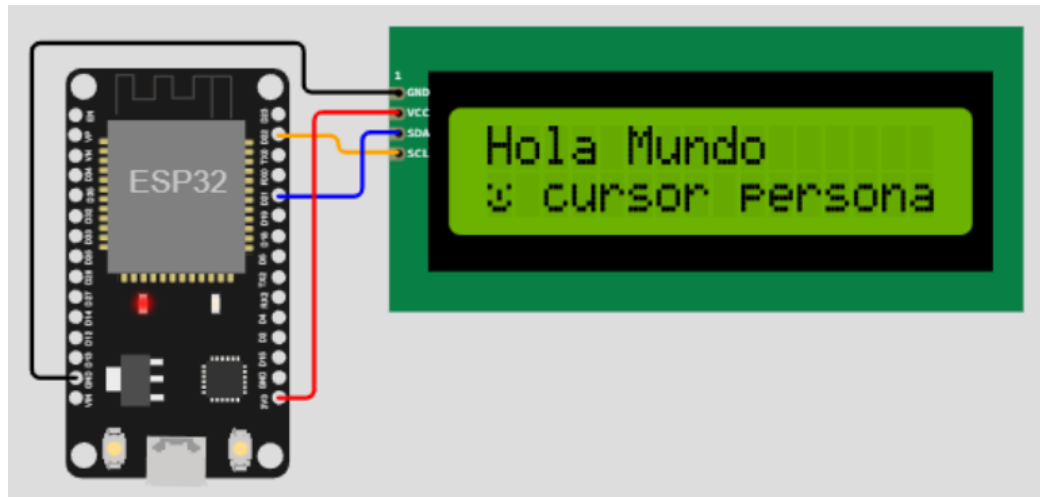
```
)
print("Create custom char ':-)')")
```

show custom char stored at location
0

```
lcd.setCursor(0,2)
lcd.print(chr(0))
lcd.setCursor(2,2)
lcd.print("cursor personalizado")
```

</pre>

Ejemplo de caracter



```
# De derecha a izquierda
lcd.clear()
lcd.setCursor(col=12, row=0)
lcd.right_to_left()
lcd.print("Hola Mundo")
```